

INFORME

Portal Educativo

Colegio Bernardo O'Higgins

Nombres :Camilo Carrasco, Carlos Donoso, Maximiliano Palma

1. Tecnologías principales

Capa	Tecnología	Propósito
Frontend	React 18 + Vite	SPA con TypeScript, TailwindCSS y Zustand para estado global
Frontend	Vitest	Test runner con @testing-library/react
Backend	Bun + Hono	Runtime ultrarrápido y framework web ligero para todos los microservicios
Backend	Drizzle ORM	ORM type-safe sobre PostgreSQL; gestiona esquemas y migraciones
Backend	Zod	Validación de DTOs y generación automática de Swagger UI
Backend	Buntest	testeo en backend con Buntest incorporado directamente con bun
Backend	JWT + bcryptjs	Autenticación HS256 stateless en el BFF; contraseñas hashadas con bcrypt
Base de datos	PostgreSQL 16	BD única con 10 schemas aislados, uno por microservicio
Infraestructura	Docker / K8s	Docker Compose para desarrollo; manifests Kubernetes para producción

2. Microservicios

El BFF (Backend for Frontend) es la única puerta de entrada del frontend: valida el JWT en /api/* y enruta al microservicio correspondiente. Cada servicio posee su propio schema en PostgreSQL.

Servicio	Puerto	Responsabilidad
BFF	3000	API Gateway. Middleware JWT global, Swagger en /docs, orquesta los 10 servicios.
Autenticación	3002	Login, registro, generación de JWT HS256, persistencia de sesiones.
Estudiantes	3001	CRUD de estudiantes. El BFF genera el JWT para su login.
Profesores	3004	CRUD de profesores.
Cursos	3005	Gestión de cursos, asignaturas y asignaciones pedagógicas.

Clases / Horario	3006	Clases por curso y bloques horarios fijos
Asistencia	3008	Registro de asistencia por clase y estudiante.
Notas	3010	Gestión de calificaciones por asignatura.
Mensajería	3009	Mensajería interna entre usuarios del sistema.
Notificaciones	3003	Notificaciones del sistema hacia usuarios.

3. Función de migración

Las migraciones son gestionadas por Drizzle ORM. El flujo es el siguiente:

- Los schemas de tabla se definen en TypeScript dentro de `src/models/` de cada microservicio.
- `drizzle-kit generate` analiza los modelos y produce archivos SQL con los cambios incrementales.
- `drizzle-kit migrate` aplica las migraciones sobre el esquema del microservicio en PostgreSQL.
- Cada servicio opera sobre su propio esquema aislado, evitando conflictos entre servicios.
- La variable `DATABASE_URL` apunta a `db:5432` dentro de Docker y a `localhost:5433` desde el host.

4. Testing

Frontend — Vitest

Usa Vitest con `jsdom` y `@testing-library/react`. Los tests están en `src/__tests__` y cubren: servicios (`auth`, `student`, `course`, `apiClient`), el store Zustand y componentes (`LoginForm`, `ProtectedRoute`, `Sidebar`, `App`).

Comando	Descripción
<code>bun test</code>	<code>watch</code> (re-ejecuta al guardar)
<code>bun run test:run</code>	Ejecución (CI/CD)
<code>bun run test:coverage</code>	Reporte de cobertura

Backend — Bun test

Cada microservicio tiene tests unitarios en `src/__tests__`. Los repositorios se inyectan como mocks, por lo que los tests no requieren base de datos. Se cubren: lógica de negocio en services y validaciones Zod en DTOs.

Comando	Descripción
<code>bun test</code>	Ejecuta los tests del microservicio
<code>bun test --coverage</code>	Incluye reporte de cobertura de código

5. Patrones de diseño

Patrón	Dónde y por qué
Backend for Frontend (BFF)	Capa única entre el frontend y los microservicios. Evita que el cliente conozca la topología interna y centraliza la autenticación.
Repository Pattern	Cada microservicio aísla el acceso a datos en <code>repositories/</code> , lo que permite mockear la base de datos en los tests unitarios.
Layered Architecture	Separación estricta <code>controller</code> → <code>service</code> → <code>repository</code> → <code>model</code> en todos los microservicios.
Database per Service	Una sola instancia de PostgreSQL, pero cada microservicio es dueño exclusivo de su esquema (autenticación, estudiantes, cursos, etc.), sin joins cruzados entre servicios.

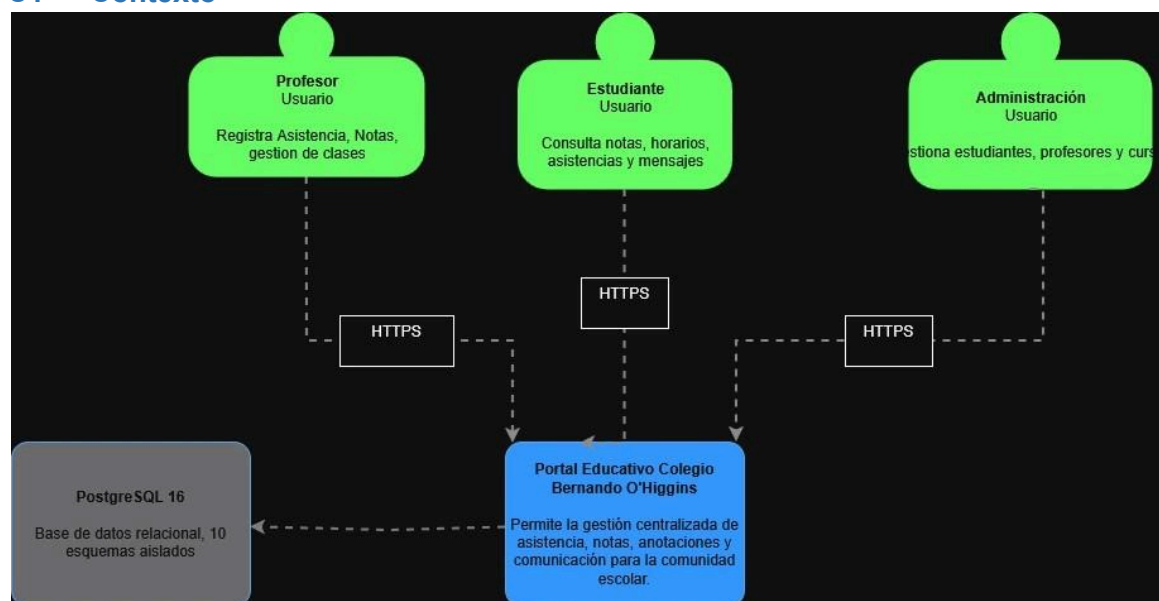
DTO + Schema Validation	Zod valida y tipa cada request antes de llegar a la capa de negocio.
Stateless Authentication	JWT HS256 verificado por firma en el BFF, sin consultar base de datos en cada request (excepto en el microservicio de Autenticación, que sí valida sesión en DB).
Screaming Architecture	El frontend organiza pages/ por dominio de negocio (auth, cursos, calificaciones, mensajería) en vez de por tipo técnico de archivo.

6. Patrones de diseño

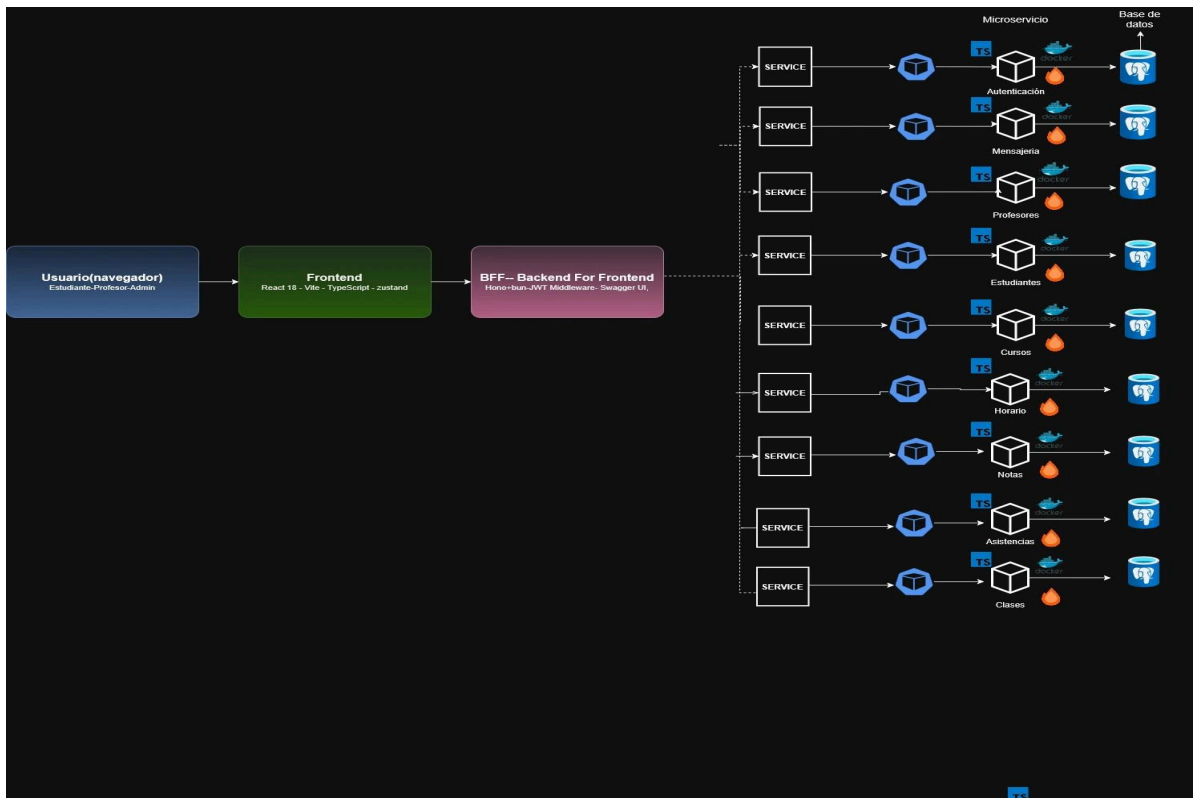
Patrón	Dónde y por qué
Backend for Frontend (BFF)	Capa única entre el frontend y los microservicios. Evita que el cliente conozca la topología interna y centraliza la autenticación.
Repository Pattern	Cada microservicio aísla el acceso a datos en repositories/, lo que permite mockear la base de datos en los tests unitarios.
Layered Architecture	Separación estricta controller → service → repository → model en todos los microservicios.
Database per Service	Una sola instancia de PostgreSQL, pero cada microservicio es dueño exclusivo de su esquema (autenticación, estudiantes, cursos, etc.), sin joins cruzados entre servicios.
DTO + Schema Validation	Zod valida y tipa cada request antes de llegar a la capa de negocio.
Stateless Authentication	JWT HS256 verificado por firma en el BFF, sin consultar base de datos en cada request (excepto en el microservicio de Autenticación, que sí valida sesión en DB).
Screaming Architecture	El frontend organiza pages/ por dominio de negocio (auth, cursos, calificaciones, mensajería) en vez de por tipo técnico de archivo.

7. Diagramas C4

C1 — Contexto



C2 — Contenedores



C3 — Componentes

